

Функции ОС по управлению памятью

Под памятью (memory) в данном случае подразумевается оперативная (основная) память компьютера. В однопрограммных операционных системах основная память разделяется на две части. Одна часть для операционной системы (резидентный монитор, ядро), а вторая – для выполняющейся в текущий момент времени программы. В многопрограммных ОС "пользовательская" часть памяти – важнейший ресурс вычислительной системы – должна быть распределена для размещения нескольких процессов, в том числе процессов ОС. Эта задача распределения выполняется операционной системой динамически специальной подсистемой управления памятью (memory management). Эффективное управление памятью жизненно важно для многозадачных систем. Если в памяти будет находиться небольшое число процессов, то значительную часть времени процессы будут находиться в состоянии ожидания ввода-вывода и загрузка процессора будет низкой.

В ранних ОС управление памятью сводилось просто к загрузке программы и ее данных из некоторого внешнего накопителя (перфоленты, магнитной ленты или магнитного диска) в ОЗУ. При этом память разделялась между программой и ОС. На рис.3 показаны три варианта такой схемы. Первая модель раньше применялась на мэйнфреймах и мини-компьютерах. Вторая схема сейчас используется на некоторых карманных компьютерах и встроенных системах, третья модель была характерна для ранних персональных компьютеров с MS-DOS.



Рис. 3. Варианты распределения памяти

С появлением мультипрограммирования задачи ОС, связанные с распределением имеющейся памяти между несколькими одновременно выполняющимися программами, существенно усложнились.

Функциями ОС по управлению памятью в мультипрограммных системах являются:

- отслеживание (учет) свободной и занятой памяти;
- первоначальное и динамическое выделение памяти процессам приложений и самой операционной системе и освобождение памяти по завершении процессов;
- настройка адресов программы на конкретную область физической памяти;

- полное или частичное вытеснение кодов и данных процессов из ОП на диск, когда размеры ОП недостаточны для размещения всех процессов, и возвращение их в ОП;
- защита памяти, выделенной процессу, от возможных вмешательств со стороны других процессов;
- дефрагментация памяти.

Перечисленные функции особого пояснения не требуют, остановимся только на задаче преобразования адресов программы при ее загрузке в ОП.

Для идентификации переменных и команд на разных этапах жизненного цикла программы используются символьные имена, виртуальные (математические, условные, логические – все это синонимы) и физические адреса (рис. 4).

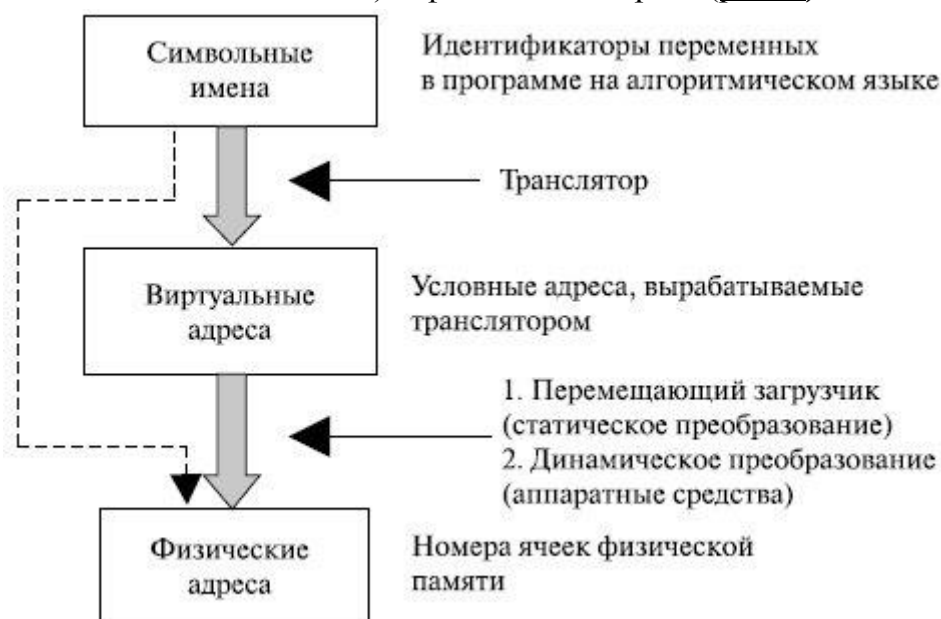


Рис. 4. Типы адресов

Символьные имена присваивает пользователь при написании программ на алгоритмическом языке или ассемблере. Виртуальные адреса вырабатывает транслятор, переводящий программу на машинный язык. Поскольку во время трансляции неизвестно, в какое место оперативной памяти будет загружена программа, транслятор присваивает переменным и командам виртуальные (условные) адреса, считая по умолчанию, что начальным адресом программы будет нулевой адрес.

Физические адреса соответствуют номерам ячеек оперативной памяти, где в действительности будут расположены переменные и команды.

Совокупность виртуальных адресов процесса называется виртуальным адресным пространством. Диапазон адресов виртуального пространства у всех процессов один и тот же и определяется разрядностью адреса процессора (для Pentium адресное пространство составляет объем, равный 2^{32} байт, с диапазоном адресов от 0000.0000_{16} до $FFFF.FFFF_{16}$).

Существует два принципиально отличающихся подхода к преобразованию виртуальных адресов в физические. В первом случае такое преобразование выполняется один раз для каждого процесса во время начальной загрузки программы в память.

Преобразование осуществляет перемещающий загрузчик на основании имеющихся у него данных о начальном адресе физической памяти, в которую предстоит загружать программу, а также информации, предоставляемой транслятором об адресно-зависимых элементах программы.

Второй способ заключается в том, что программа загружается в память в виртуальных адресах. Во время выполнения программы при каждом обращении к памяти операционная система преобразует виртуальные адреса в физические.

6.3. Распределение памяти

Существует ряд базовых вопросов управления памятью, которые в различных ОС решаются по-разному. Например, следует ли назначать каждому процессу одну непрерывную область физической памяти или можно выделять память участками? Должны ли сегменты программы, загруженные в память, находиться на одном месте в течение всего периода выполнения процесса или их можно время от времени сдвигать? Что делать, если сегменты программы не помещаются в имеющуюся память? Как сократить затраты ресурсов системы на управление памятью? Имеется и ряд других не менее интересных проблем управления памятью.

Ниже приводится классификация методов распределения памяти, в которой выделено два класса методов – с перемещением сегментов процессов между ОП и ВП (диском) и без перемещения, т.е. без привлечения внешней памяти (рис. 5). Данная классификация учитывает только основные признаки методов. Для каждого метода может быть использовано несколько различных алгоритмов его реализации.



Рис. 5. Классификация методов распределения памяти

На рис. 6 показаны два примера фиксированного распределения. Одна возможность состоит в использовании разделов одинакового размера. В этом случае любой процесс, размер которого не превышает размера раздела, может быть загружен в любой доступный раздел. Если все разделы заняты и нет ни одного процесса в состоянии

готовности или работы, ОС может выгрузить процесс из любого раздела и загрузить другой процесс, обеспечивая тем самым процессор работой.

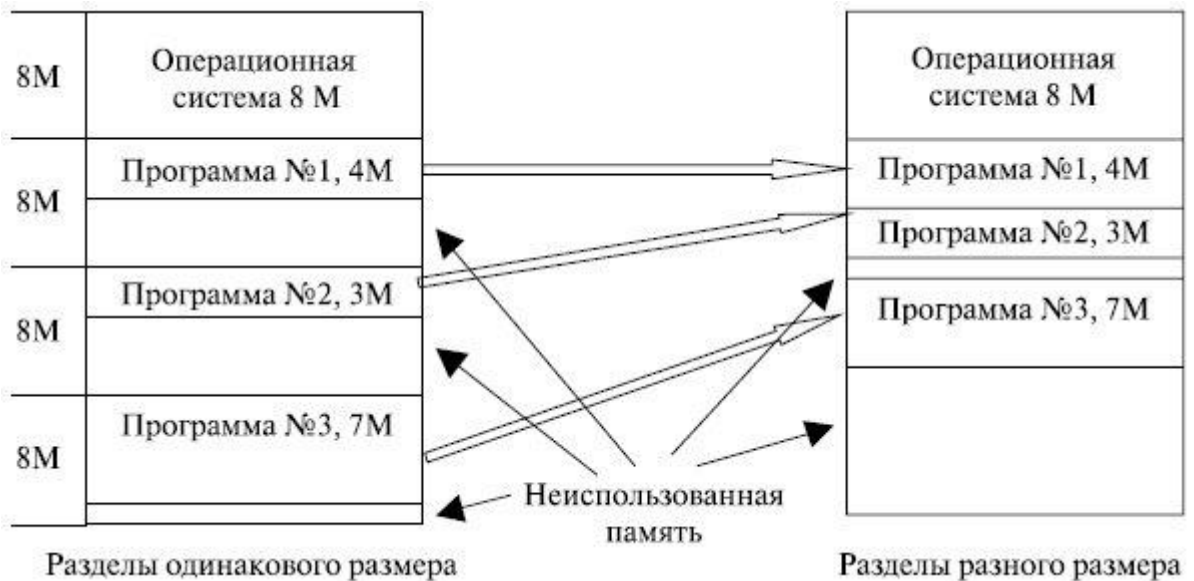


Рис. 6 Варианты фиксированного распределения памяти

При использовании разделов с одинаковым размером имеются две проблемы.

1. Программа может быть слишком велика для размещения в разделе. В этом случае программист должен разрабатывать программу, использующую оверлеи, чтобы в любой момент времени требовался только один раздел памяти. Когда требуется модуль, отсутствующий в данный момент в ОП, пользовательская программа должна сама его загрузить в раздел памяти программы. Таким образом, в данном случае управление памятью во многом возлагается на программиста.

2. Использование ОП крайне неэффективно. Любая программа, независимо от ее размера, занимает раздел целиком. При этом могут оставаться неиспользованные участки памяти большого размера. Этот феномен появления неиспользованной памяти называется внутренней фрагментацией (internal fragmentation).

Бороться с этими трудностями (хотя и не устранить полностью) можно посредством использования разделов разных размеров. В этом случае программа размером до 8 Мбайт может обойтись без оверлеев, а разделы малого размера позволяют уменьшить внутреннюю фрагментацию при загрузке небольших программ.

В том случае, когда разделы имеют одинаковый размер, размещение процессов тривиально – в любой свободный раздел. Если все разделы заняты процессами, которые не готовы к немедленной работе, любой из них может быть выгружен для освобождения памяти для нового процесса.

Когда разделы имеют разные размеры, есть два возможных подхода к назначению процессов разделам памяти. Простейший путь состоит в том, чтобы каждый процесс размещался в наименьшем разделе, способном вместить данный процесс (в этом случае

в задании пользователя указывался размер требуемой памяти). При таком подходе для каждого раздела требуется очередь планировщика, в которой хранятся выгруженные из памяти процессы, предназначенные для данного раздела памяти. Достоинство такого способа в возможности распределения процессов между разделами ОП так, чтобы минимизировать внутреннюю фрагментацию.

Недостаток заключается в том, что отдельные очереди для разделов могут привести к неоптимальному распределению памяти системы в целом. Например, если в некоторый момент времени нет ни одного процесса размером от 7 до 12 Мбайт, то раздел размером 12 Мбайт будет пустовать, в то время как он мог бы использоваться меньшими процессами. Поэтому более предпочтительным является использование одной очереди для всех процессов. В момент, когда требуется загрузить процесс в ОП, выбирается наименьший доступный раздел, способный вместить данный процесс.

В целом можно отметить, что схемы с фиксированными разделами относительно просты, предъявляют минимальные требования к операционной системе; накладные расходы работы процессора на распределение памяти невелики. Однако у этих схем имеются серьезные недостатки.

1. Количество разделов, определенное в момент генерации системы, ограничивает количество активных процессов (т.е. уровень мультипрограммирования).

2. Поскольку размеры разделов устанавливаются заранее во время генерации системы, небольшие задания приводят к неэффективному использованию памяти. В средах, где заранее известны потребности в памяти всех задач, применение рассмотренной схемы может быть оправдано, но в большинстве случаев эффективность этой технологии крайне низка.

Для преодоления сложностей, связанных с фиксированным распределением, был разработан альтернативный подход, известный как динамическое распределение. В свое время этот подход был применен фирмой IBM в операционной системе для мэйнфреймов в OS/MVT (мультипрограммирование с переменным числом задач – Multiprogramming With a Variable number of Tasks). Позже этот же подход к распределению памяти использован в ОС ЕС ЭВМ.

При динамическом распределении образуется переменное количество разделов переменной длины. При размещении процесса в основной памяти для него выделяется строго необходимое количество памяти. В качестве примера рассмотрим использование 64 Мбайт (рис. 6) основной памяти. Изначально вся память пуста, за исключением области, задействованной ОС. Первые три процесса загружаются в память, начиная с адреса, где заканчивается ОС, и используют столько памяти, сколько требуется данному процессу. После этого в конце ОП остается свободный участок памяти, слишком малый для размещения четвертого процесса. В некоторый момент времени все процессы в памяти оказываются неактивными, и операционная система выгружает второй процесс, после чего остается достаточно памяти для загрузки нового, четвертого процесса.

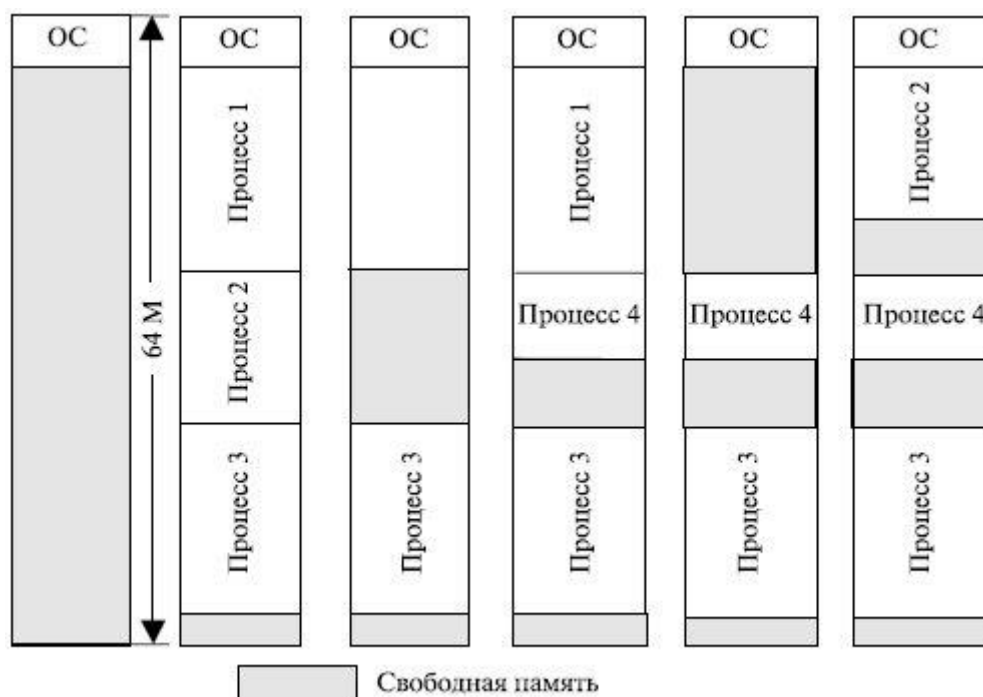


Рис. 7. Вариант использования памяти

Поскольку процесс 4 меньше процесса 2, появляется еще свободный участок памяти. После того как в некоторый момент времени все процессы оказались неактивными, но стал готовым к работе процесс 2, свободного места в памяти для него не находится, а ОС вынуждена выгрузить процесс 1, чтобы освободить необходимое место и разместить процесс 2 в ОП. Как показывает данный пример, этот метод хорошо начинает работу, но плохо продолжает. В конечном счете, он приводит к наличию множества мелких свободных участков памяти, в которых нет возможности разместить какой-либо новый процесс. Это явление называется внешней фрагментацией (external fragmentation), что отражает тот факт, что сильно фрагментированной становится память, внешняя по отношению ко всем разделам.

Один из методов преодоления внешней фрагментации – уплотнение (compaction) процессов в ОП. Осуществляется это перемещением всех занятых участков так, чтобы вся свободная память образовала единую свободную область. В дополнение к функциям, которые ОС выполняет при распределении памяти динамическими разделами, в данном случае она должна еще время от времени копировать содержимое разделов из одного места в другое, корректируя таблицы свободных и занятых областей. Эта процедура называется уплотнением или сжатием.

Перечислим функции операционной системы по управлению памятью в этом случае.

1. Перемещение всех занятых участков в сторону старших или младших адресов при каждом завершении процесса или для вновь создаваемого процесса в случае отсутствия раздела достаточного размера.
2. Коррекция таблиц свободных и занятых областей.

3. Изменение адресов команд и данных, к которым обращаются процессы при их перемещении в памяти, за счет использования относительной адресации.

4. Аппаратная поддержка процесса динамического преобразования относительных адресов в абсолютные адреса основной памяти.

5. Защита памяти, выделяемой процессу, от взаимного влияния других процессов.

Уплотнение может выполняться либо при каждом завершении процесса, либо только тогда, когда для вновь создаваемого процесса нет свободного раздела достаточного размера. В первом случае требуется меньше вычислительной работы при корректировке таблиц свободных и занятых областей, а во втором – реже выполняется процедура сжатия.

Так как программа перемещается по оперативной памяти в ходе своего выполнения, в данном случае невозможно выполнить настройку адресов с помощью перемещающего загрузчика. Здесь более подходящим оказывается динамическое преобразование адресов. Достоинствами распределения памяти перемещаемыми разделами являются эффективное использование оперативной памяти, исключение внутренней и внешней фрагментации, недостатком – дополнительные накладные расходы ОС.

При использовании фиксированной схемы распределения процесс всегда будет назначаться одному и тому же разделу памяти после его выгрузки и последующей загрузке в память. Это позволяет применять простейший загрузчик, который замещает при загрузке процесса все относительные ссылки абсолютными адресами памяти, определенными на основе базового адреса загруженного процесса.

Ситуация усложняется, если размеры разделов равны (или неравны) и существует единая очередь процессов, – процесс по ходу работы может занимать разные разделы. Такая же ситуация возможна и при динамическом распределении. В этих случаях расположение команд и данных, к которым обращается процесс, не является фиксированным и изменяется всякий раз при выгрузке, загрузке или перемещении процесса. Для решения этой проблемы в программах используются относительные адреса. Это означает, что все ссылки на память в загружаемом процессе даются относительно начала этой программы. Таким образом, для корректной работы программы требуется аппаратный механизм, который бы транслировал относительные адреса в физические в процессе выполнения команды, обращающейся к памяти.

Применяемый обычно способ трансляции показан на рис. 8. Когда процесс переходит в состояние выполнения, в специальный регистр процесса, называемый базовым, загружается начальный адрес процесса в основной памяти. Кроме того, используется "граничный" (bounds) регистр, в котором содержится адрес последней ячейки программы. Эти значения заносятся в регистры при загрузке программы в основную память. При выполнении процесса относительные адреса в командах обрабатываются процессором в два этапа. Сначала к относительному адресу прибавляется значение базового регистра для получения абсолютного адреса. Затем

полученный абсолютный адрес сравнивается со значением в граничном регистре. Если полученный абсолютный адрес принадлежит данному процессу, команда может быть выполнена. В противном случае генерируется соответствующее данной ошибке прерывание.

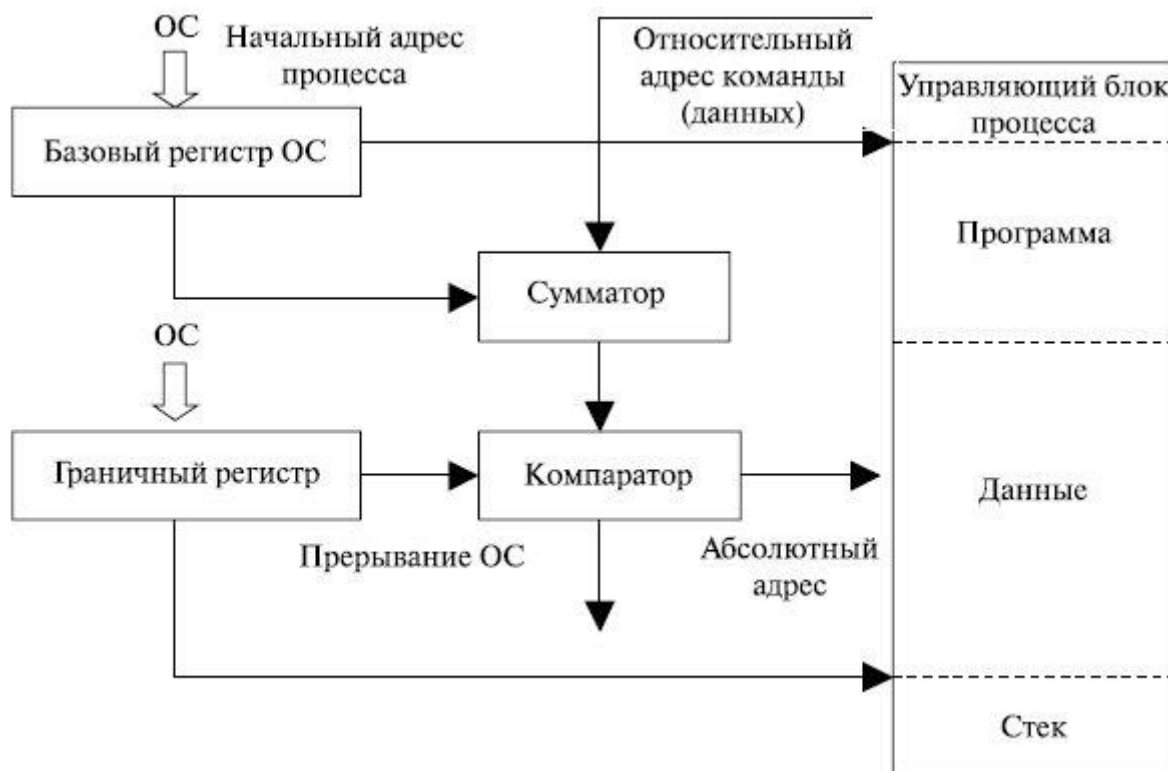


Рис. 8. Преобразование адресов